# Communication

| | |
|---|---|
| **Date** | 28/06/2026 |
| **Team ID** | Not Assigned |
| **Project Name** | Rising Waters: A Machine Learning Approach to Flood Prediction |
| **Maximum Marks** | 1 Mark |

### Communication Plan:

Effective communication is essential for a successful project demonstration. This document outlines the communication strategy used within the Rising Waters team and with stakeholders throughout the project lifecycle, including how updates, issues, and feedback were managed.

| S.No | Communication Type | Frequency | Channel / Tool | Participants | Purpose |
|---|---|---|---|---|---|
| 1 | **Team Standup** | Daily | WhatsApp Group Chat | All 5 Team Members | Quick daily check-in on each member's progress, blockers, and tasks for the day. Ensured everyone stayed aligned throughout the 4-day sprint. |
| 2 | **Progress Update** | Weekly (End of Sprint) | Google Meet / In-Person | All 5 Team Members | Reviewed completed sprint tasks, shared notebook outputs and EDA plots, discussed model accuracy results, and planned next steps for the following day. |
| 3 | **Issue / Bug Discussion** | As Needed | WhatsApp + VS Code LiveShare | Affected Members | Resolved technical issues such as model save errors, Flask route bugs, Procfile configuration, Git push conflicts, and Render.com deployment issues in real-time. |
| 4 | **Stakeholder Review** | Bi-Weekly | Google Meet + GitHub Repository | Team + Mentor/Guide | Demonstrated project progress to the project guide — shared EDA visualizations, model comparison results, and live Flask application demo for feedback and approval. |
| 5 | **Final Demo Rehearsal** | Once (27/06/2026) | In-Person + Screen Share | All 5 Team Members | Full end-to-end dry run of the project demonstration — running notebooks, showing web application, testing Render.com deployment, and rehearsing Q&A; responses. |
| 6 | **GitHub Code Review** | After Each Commit | GitHub Repository | All 5 Team Members | Each team member reviewed code commits before merging to main branch. Ensured code quality, correct file structure, and verified Render.com auto-deployment triggered successfully. |

### Communication Challenges & Resolutions:

| S.No | Challenge Faced | Resolution / Action Taken |
|---|---|---|
| 1 | Git push conflict when two team members committed changes to app.py simultaneously, causing merge errors on the main branch. | Resolved by assigning separate file ownership — Gowthamsai Setti handled app.py and Flask routes while Rishitha Padaga managed HTML templates. Used git pull before every push to prevent future conflicts. |
| 2 | Render.com deployment failed initially because floods.save and transform.save were not committed to the GitHub repository, causing a FileNotFoundError at startup. | Both .save files were added to the GitHub repository using git add floods.save transform.save and committed. Verified on Render.com logs that files loaded correctly on next auto-deploy. |
| 3 | Team coordination was difficult during the model building phase as three members were working on different classifiers (KNN, Random Forest, XGBoost) simultaneously in separate notebooks. | Resolved by creating a shared Google Drive folder where each member uploaded their notebook outputs and accuracy screenshots daily. Daily WhatsApp standups ensured everyone's results were compared and the best model was agreed upon as XGBoost. |